

University of Calgary

Department of Electrical and Software Engineering

ENSF 694 - Data Structures and Algorithms

Term Project

Campus Navigation and Event Management System

Submitted by:

Peter Osaade (UCID: 30279322)
Varsy Gevorkyan (UCID: 30251736)
GodsFavour Akande (UCID: 30321043)

Group 5

Submission Date: August 19, 2026

1. Brief

This report covers the Campus Navigation and Event Management System, a C++ console application. The system models a campus as a graph of buildings and paths, and supports 7 features (includes the bonus).

2. Design Decisions

For each feature: the data structure used, why it fits, and one alternative that was considered and rejected.

2.1 Campus Map and Shortest-Path Navigation

Data structure / algorithm: Weighted undirected graph as an adjacency list (Graph class), with Dijkstra's algorithm using a custom binary min-heap.

Why: A campus map is naturally a graph of buildings connected by paths, so this was the most direct way to represent it. Dijkstra's algorithm is the standard way we learned in class to find a shortest path when all the weights are positive, so it was the natural fit here.

Alternative considered: We also thought about an adjacency matrix, but a list felt simpler once we noticed how many building pairs have no direct path between them at all.

2.2 Route History and Navigation Undo

Data structure / algorithm: A capped LIFO stack (NavigationHistory), built on a vector, with a max depth of 50 (above the 10 asked).

Why: A stack fits how undo works: the last route you looked at is the one you go back to first, so this was the simplest match for the problem. We also put a cap on how many routes it remembers, since letting it grow with no limit at all over a long session felt like it could get out of hand.

Alternative considered: We considered not capping the history at all, but that seemed risky for a session that runs for a long time.

2.3 Room and Event Booking System

Data structure / algorithm: A self-balancing AVL tree (BookingAVL), keyed by start time then booking ID, with standard rotations after every insert and remove.

Why: We were worried that if bookings get added in date order, which is pretty likely in real use, a plain tree could end up basically as slow as a list. A self-balancing tree avoids that. Sorting by start time also made the range and day queries much easier to write, since we can stop looking once we're outside the requested window instead of checking every booking.

Alternative considered: A hash table keyed by booking ID would be fast for looking things up by ID, but it does not keep things in order, so range and day queries would need to check everything and sort it after. That felt like the wrong tool for this part.

2.4 Priority-Based Service Queue

Data structure / algorithm: A binary max-heap (ServicePriorityQueue) built on a vector, ordered by priority level and then by arrival order for ties.

Why: A heap was something we understood well from lecture, and it naturally keeps the highest-priority request on top so serving the next one is quick. We also broke ties by arrival order so two requests with the same priority do not end up served in a random order.

Alternative considered: We thought about just keeping a sorted list and inserting each new request in the right spot, but that means shifting things around every time a request arrives, which felt wasteful once requests keep coming in.

2.5 Fast Building and Resource Lookup

Data structure / algorithm: A separate-chaining hash table (BuildingHashTable) with 30+ buckets and a simple rolling hash function on the building ID.

Why: A hash table was the simplest way to get fast lookups by ID, and it is the tool this kind of problem calls for. We used a prime number of buckets since that is a common trick to spread entries out more evenly and avoid clustering.

Alternative considered: A search tree would also work, but it did not seem necessary since we never need the buildings in sorted order, only a quick lookup by ID.

2.6 Incoming Request Processing

Data structure / algorithm: A circular array-based FIFO queue (RequestQueue) that doubles its capacity when full.

Why: A queue is the obvious structure for handling things in the order they arrive. We used a circular array instead of a linked list mostly because it was simpler to implement and does not need a new memory allocation for every single request.

Alternative considered: A linked-list queue would also work here, but felt like more overhead than we needed for this part.

2.7 (Bonus) Balanced Event Index

Data structure / algorithm: Reuses the AVL tree from Section 2.3.

Why: Since the same tree already rebalances itself after every insert, it satisfies the bonus requirement without building a second structure. `isBalanced()` and `printTree()` let us check, and show, that the balance rule actually holds after a batch of insertions.

Alternative considered: Not applicable, this feature reuses the structure from Section 2.3, would be silly to build something twice instead of just reuse.

3. Complexity Analysis

A short summary of the main operation for each feature.

Feature	Main Operation	Big O
Shortest path	Dijkstra with min-heap	$O(n \log n)$
Navigation undo	push / undo	$O(1)$
Booking index (AVL)	insert / remove	$O(\log n)$
Service priority queue	enqueue / dequeue	$O(\log n)$

Feature	Main Operation	Big O
Building lookup (hash table)	insert / find / remove	O(1)
Request pipeline	enqueue / dequeue	O(1)

4. Challenges and Lessons Learned

- Getting the AVL rotations right after both insert and delete took careful testing, especially inserting bookings in date order to check the tree stayed balanced.
- Sorting the AVL by start time instead of booking ID made range and day queries fast, but meant ID lookup had to fall back to a full scan. This was a trade-off, not a bug.
- Breaking ties in the priority queue by arrival order took an extra field on each request, but was needed so equal-priority requests are served fairly.
- Putting all six structures behind one CampusSystem class and one console menu made it easier to test each feature on its own and catch integration bugs early.

5. Contributions

Group Member	UCID	Primary Contribution(s)
Peter Osaade	30279322	Finding shortest route, Undo last navigation, Find Building/Resource, Add/Remove Booking, and this report.
Varsy Gevorkyan	30251736	Submit Service Request, Serve Highest-Priority Request, Enqueue Incoming Request, Process Next Incoming Request, and this report.
GodsFavour Akande	30321043	Show Campus Graph, Show AVL Booking Tree, Run Required Scenarios, Report Creation, README, and this report.

6. Demo Scenarios and Screenshots

The scenario below comes from running `./campus_app` and choosing option 13 ("Run required demo scenarios") from the main menu.

```
peter@PETERs-MacBook-Pro Project_Group_Work % g++ -std=c++17 -Wall -Wextra -pedantic src/*.cpp -o campus_app
peter@PETERs-MacBook-Pro Project_Group_Work % ./campus_app
Loaded 16 campus nodes and 28 pathways.
Loaded 100 bookings.
```

```
===== CAMPUS SYSTEM =====
```

1. Find shortest route
2. Undo last navigation
3. Find building/resource
4. Add booking
5. Remove booking
6. Query bookings by time range
7. Submit service request
8. Serve highest-priority request
9. Enqueue incoming request
10. Process next incoming request
11. Show campus graph
12. Show AVL booking tree
13. Run required demo scenarios
0. Exit

```
=====
```

Select an option: 13

```
===== DEMO SCENARIOS =====
```

[1] Shortest path queries

ICT -> SU -> RES

Total travel time: 8 minutes

LIB -> SCI -> GYM

Total travel time: 8 minutes

[2] Undo navigation

Undid route: LIB -> GYM

Previous origin restored to: LIB

[3] Booking range query

#3 ENG-201 | 2026-08-17 10:00 - 11:00 | Software Design Lab

#4 LIB-301 | 2026-08-17 11:00 - 12:00 | Project Meeting

#5 SCI-115 | 2026-08-17 12:00 - 13:00 | Tutorial

#6 MFH-160 | 2026-08-17 13:00 - 14:00 | Research Seminar

#7 EEEL-210 | 2026-08-17 14:00 - 15:00 | Club Meeting

Returned 5 booking(s).

[4] Priority queue demo

Serving request #2 priority=3 Water leak

Serving request #4 priority=3 Network outage

Serving request #1 priority=2 Printer jam

Serving request #3 priority=1 Replace whiteboard marker

[5] Fast lookup demo

Lookup ICT: FOUND

Lookup DOES_NOT_EXIST: NOT FOUND

[6] FIFO incoming request pipeline

Processed #1 NAV Request_1

Processed #2 SERVICE Request_2

Processed #3 NAV Request_3

Processed #4 SERVICE Request_4

Processed #5 NAV Request_5

Processed #6 SERVICE Request_6

Processed #7 NAV Request_7

Processed #8 SERVICE Request_8

Processed #9 NAV Request_9

Processed #10 SERVICE Request_10

Processed #11 NAV Request_11

Processed #12 SERVICE Request_12

Processed #13 NAV Request_13

Processed #14 SERVICE Request_14

Processed #15 NAV Request_15

Processed #16 SERVICE Request_16

Processed #17 NAV Request_17

Processed #18 SERVICE Request_18

Processed #19 NAV Request_19

Processed #20 SERVICE Request_20

[Bonus] AVL booking index

Booking count: 100

AVL balanced: YES

The scenario below comes from running ./campus_app and choosing option 1 ("Find shortest route").

```
peter@PETERs-MacBook-Pro Project_Group_Work % g++ -std=c++17 -Wall -Wextra -pedantic src/*.cpp -o campus_app
peter@PETERs-MacBook-Pro Project_Group_Work % ./campus_app
Loaded 16 campus nodes and 28 pathways.
Loaded 100 bookings.

===== CAMPUS SYSTEM =====
1. Find shortest route
2. Undo last navigation
3. Find building/resource
4. Add booking
5. Remove booking
6. Query bookings by time range
7. Submit service request
8. Serve highest-priority request
9. Enqueue incoming request
10. Process next incoming request
11. Show campus graph
12. Show AVL booking tree
13. Run required demo scenarios
0. Exit
=====
Select an option: 1
Source building ID: ENA
Destination building ID: LIB
Route: ENA -> ENG -> SCI -> LIB
Total travel time: 8 minutes
```

The scenario below comes from running ./campus_app and choosing option 2 from the main menu.

```
===== CAMPUS SYSTEM =====
1. Find shortest route
2. Undo last navigation
3. Find building/resource
4. Add booking
5. Remove booking
6. Query bookings by time range
7. Submit service request
8. Serve highest-priority request
9. Enqueue incoming request
10. Process next incoming request
11. Show campus graph
12. Show AVL booking tree
13. Run required demo scenarios
0. Exit
=====
Select an option: 2
Undid route ENA -> LIB
Returned to previous origin: ENA
```

The scenario below comes from running ./campus_app and choosing option 11 ("Show campus graph") from the main menu.

```
Select an option: 11
ICT -> LIB(4), ENG(6), SU(3), PARK(7)
LIB -> ICT(4), SCI(3), ADMIN(4), TFDL(2)
ENG -> ICT(6), SCI(2), MFH(2), ENA(3), EEEL(4)
SU -> ICT(3), RES(5), MAC(2), TFDL(4)
PARK -> ICT(7), RES(4), GYM(6)
SCI -> LIB(3), ENG(2), GYM(5), BIO(3)
ADMIN -> LIB(4), BIO(3), MAC(3)
TFDL -> LIB(2), SU(4), MAC(2)
GYM -> SCI(5), PARK(6), RES(3), OVAL(2)
BIO -> SCI(3), ADMIN(3), EEEL(5)
MFH -> ENG(2), EEEL(2), ENA(2)
ENA -> ENG(3), MFH(2), EEEL(2)
EEEL -> ENG(4), MFH(2), BIO(5), ENA(2)
RES -> SU(5), PARK(4), GYM(3), OVAL(4)
MAC -> SU(2), ADMIN(3), TFDL(2)
OVAL -> GYM(2), RES(4)
```

Appendix A: AI Use Disclosure

- The `campus_map.txt` and `bookings.txt` files were generated using AI as this is a tedious and non-coding exercise .
- Additionally, some of the commenting was edited with AI to make more sense.
- Also used when faced with issues and bugs to figure out resolution - for issues like the one discussed below.

Appendix B: Running Issue Note

Note, that when running using command `g++ -Wall src/*.cpp -o campus_system` some members had no errors, while others would see a screen full of warnings.

AI suggested use: `g++ -std=c++17 -Wall -Wextra -pedantic src/*.cpp -o campus_app`

Do so if you see this:

```
src/Graph.h:13:22: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    double totalCost = 0.0;
                       ^
In file included from src/main.cpp:4:
In file included from src/CampusSystem.h:10:
src/BookingAVL.h:12:14: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    int year = 0;
               ^
src/BookingAVL.h:13:15: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    int month = 0;
                ^
src/BookingAVL.h:14:13: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    int day = 0;
               ^
src/BookingAVL.h:15:14: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    int hour = 0;
                ^
src/BookingAVL.h:16:16: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    int minute = 0;
                 ^
src/BookingAVL.h:29:19: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    int bookingID = 0;
                  ^
src/BookingAVL.h:79:37: warning: deleted function definitions are a C++11 extension [-Wc++11-extensions]
    BookingAVL(const BookingAVL&) = delete;
                                   ^
src/BookingAVL.h:80:48: warning: deleted function definitions are a C++11 extension [-Wc++11-extensions]
    BookingAVL& operator=(const BookingAVL&) = delete;
                                                ^
In file included from src/main.cpp:4:
In file included from src/CampusSystem.h:11:
src/ServicePriorityQueue.h:10:19: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    int requestID = 0;
                  ^
src/ServicePriorityQueue.h:12:18: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    int priority = 1; // 3 = Emergency, 2 = Standard, 1 = Low
                   ^
src/ServicePriorityQueue.h:13:28: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    long long arrivalOrder = 0;
                           ^
In file included from src/main.cpp:4:
In file included from src/CampusSystem.h:12:
src/RequestQueue.h:10:19: warning: default member initializer for non-static data member is a C++11 extension [-Wc++11-extensions]
    int requestID = 0;
                  ^
src/main.cpp:165:13: warning: 'auto' type specifier is a C++11 extension [-Wc++11-extensions]
    auto results = campus.bookingsInRange(
    ^
src/main.cpp:170:24: warning: 'auto' type specifier is a C++11 extension [-Wc++11-extensions]
    for (const auto& b : results) printBooking(b);
               ^
src/main.cpp:170:32: warning: range-based for loop is a C++11 extension [-Wc++11-extensions]
    for (const auto& b : results) printBooking(b);
                           ^
19 warnings generated.
```